

NOTICE

- Deadlines
 - (5/13 11:59 PM) Programming assignment 3
 - Midterm Quiz 3 will be released on the 6th

CS 374: OPERATING SYSTEMS I

PART III – SOCKETS

M/W 12:00 – 1:50 PM (LINC #200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



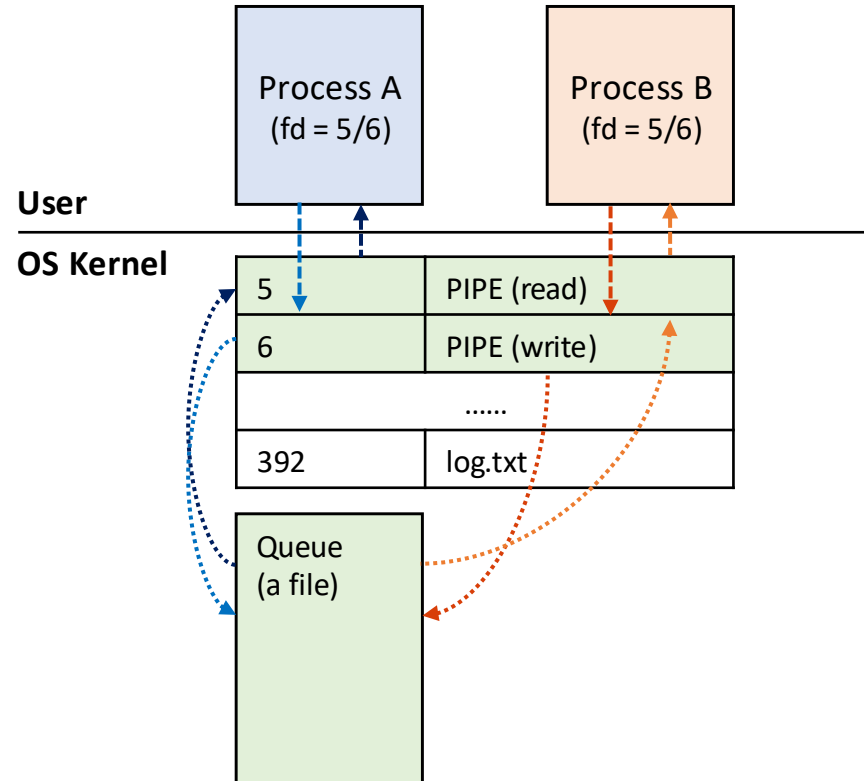
TRUE AI
Trustworthy and Responsible AI

TOPICS FOR TODAY

- Part III: Sockets
 - Provide abstraction
 - Why is PIPE-based IPC not enough?
 - What is a socket?
 - Offer standard interface
 - How do we use sockets?
 - How do we handle multiple concurrent connections?
 - Manage resources
 - (Not in this lecture) How does the OS manages socket internals?

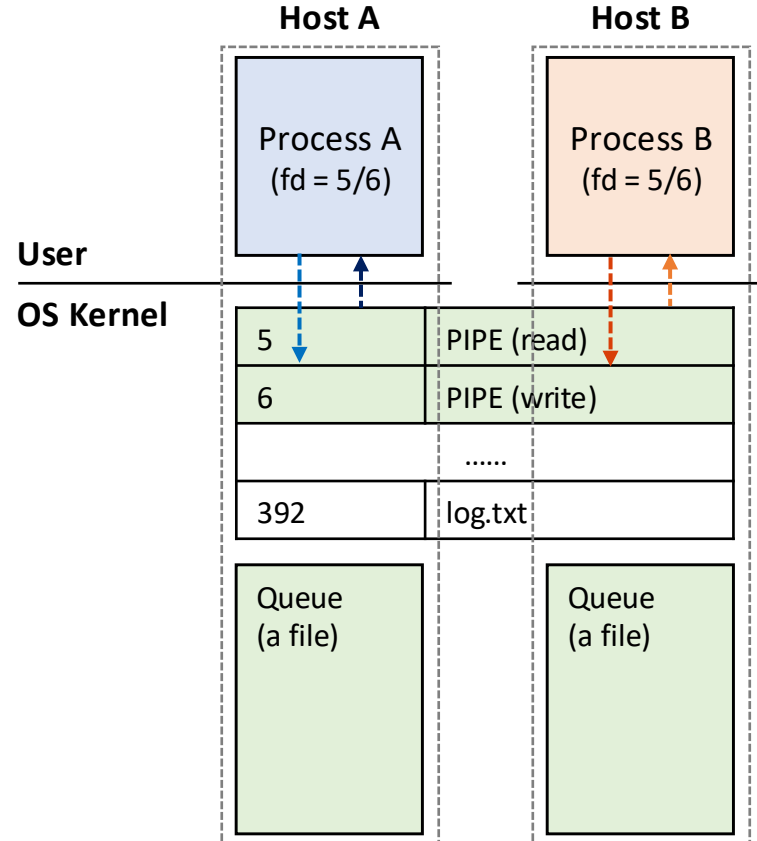
MOTIVATION: WHY PIPE-BASED IPC IS NOT ENOUGH?

- Pipe only support IPCs
 - What if Proc A and B are running on different hosts (or machines)?



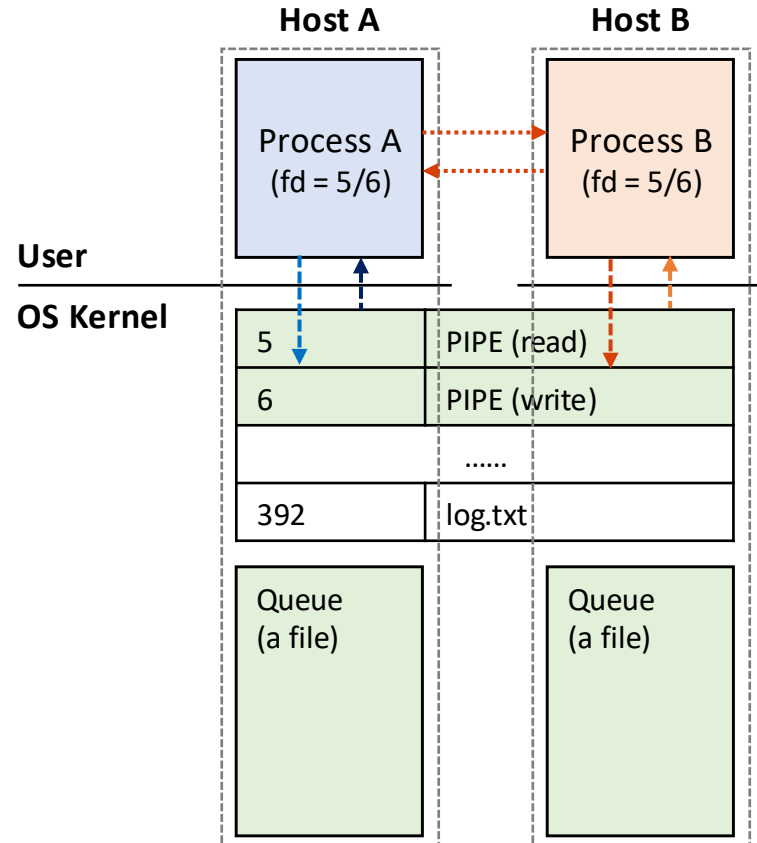
MOTIVATION: SOLUTION DESIGN

- Pipe only support IPCs
 - What if Proc A and B are running on different hosts (or machines)?
- **Solution approach**
 - Each process has its own queue



MOTIVATION: RPC DESIGN

- Pipe only support IPCs
 - What if Proc A and B are running on different hosts (or machines)?
- **Solution approach**
 - Each process has its own queue
 - Design a communication *protocol(s)*



MOTIVATION: RPC PROTOCOL DESIGN

- **Caller (You)**

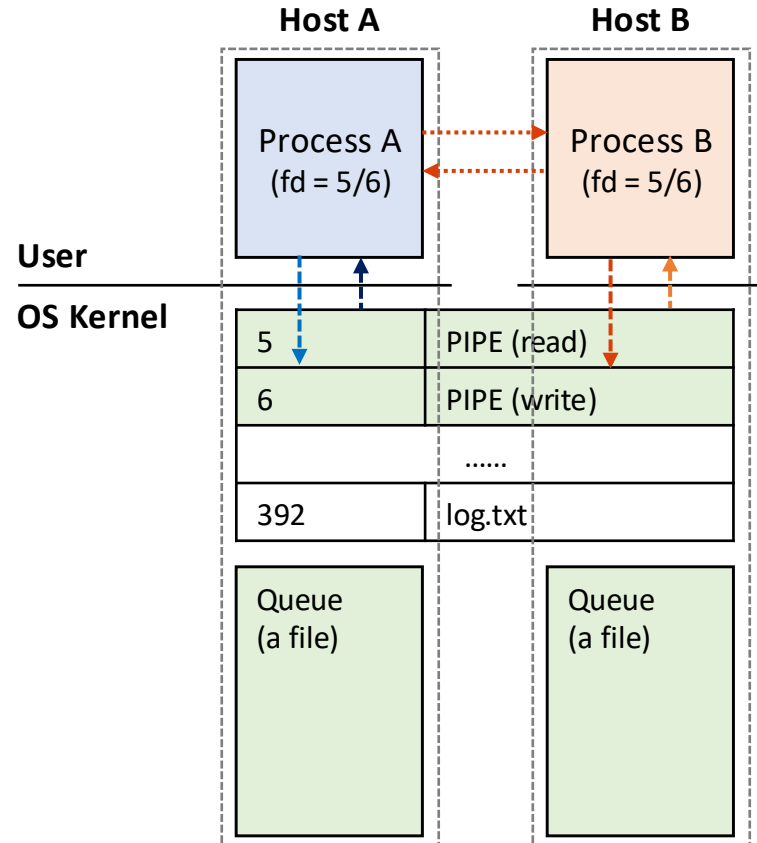
- Open up your phone
- Search a restaurant's phone number
- Call and wait
- I'd like to have a table for two today at 7 pm
- John Doe
- 123-456-7890
- Thank you
- Hang up

- **Callee (XYZ Restaurant)**

- Thank you for calling XYZ. How can I help you?
- Two at 7 pm. Yes, we have a table.
- May I have the name on the reservation?
- and a phone number?
- Today, 7 pm today, John Doe. You're all set.
- Thank you. See you soon.
- Bye

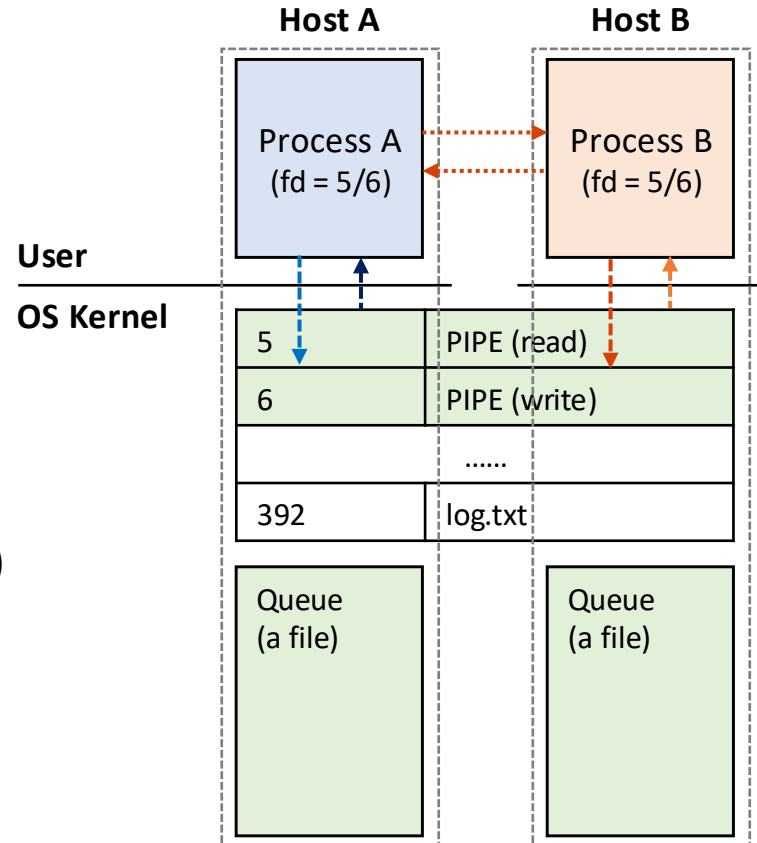
MOTIVATION: RPC PROTOCOL DESIGN

- Pipe only support IPCs
 - What if Proc A and B are running on different hosts (or machines)?
 - What if there are multiple hosts?
- **Solution approach**
 - Each process has its own queue
 - Design a communication *protocol(s)*



MOTIVATION: RPC PROTOCOL DESIGN

- Pipe only support IPCs
 - What if Proc A and B are running on different hosts (or machines)?
 - What if there are multiple hosts?
- **Solution approach**
 - Each process has its own queue
 - Design a communication *protocol(s)*
 - Require an *address* for each host (like a phone number for the restaurant)

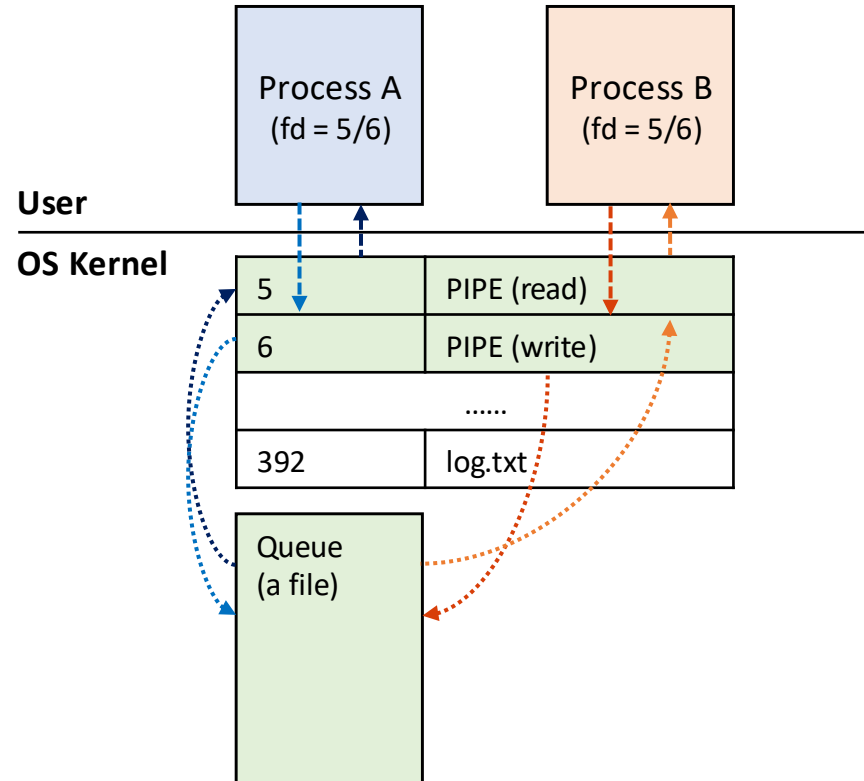


PROVIDE ABSTRACTION: SOCKET

- Socket
 - **Definition:** an *abstract* structure for sending and receiving data
 - **TL; DR:** a *bi-directional* pipe

REVISIT: PIPE IS UNIDIRECTIONAL

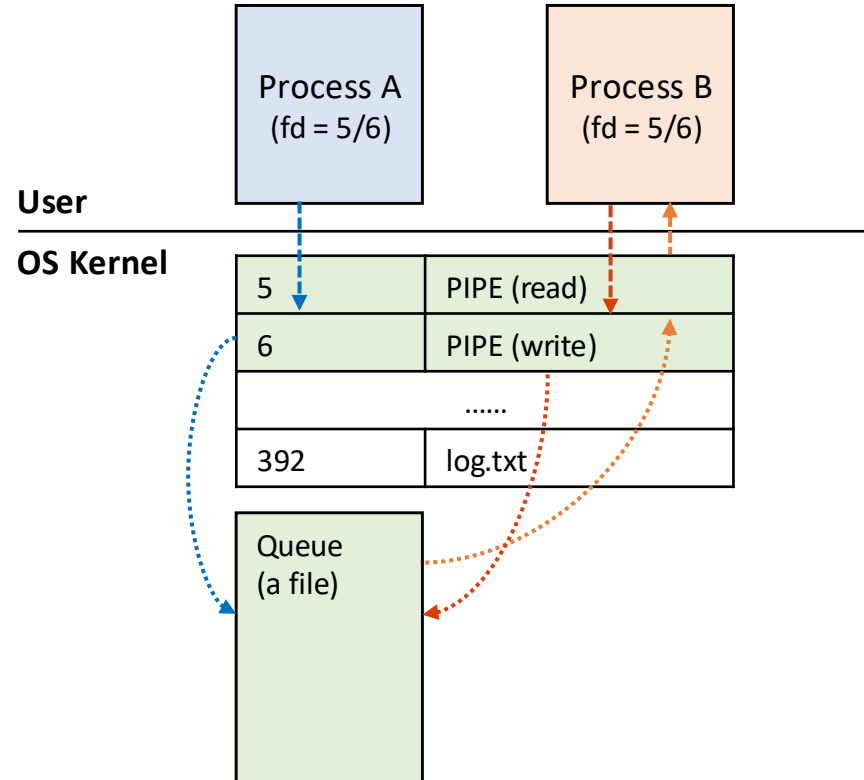
- Rules
 - Proc A opens a pipe
 - Proc A forks and creates Proc B



REVISIT: PIPE IS UNIDIRECTIONAL – CONT'D

- Rules

- Proc A opens a pipe
- Proc A forks and creates Proc B
- Proc A write – Queue [Hello]
- Proc B write – Queue [Hello | World]
- Proc B `read()` returns “Hello”
- Due to this, we set the roles
 - Proc A – only `read()` data
 - Proc B – only `write()` data



PROVIDE ABSTRACTION: SOCKET

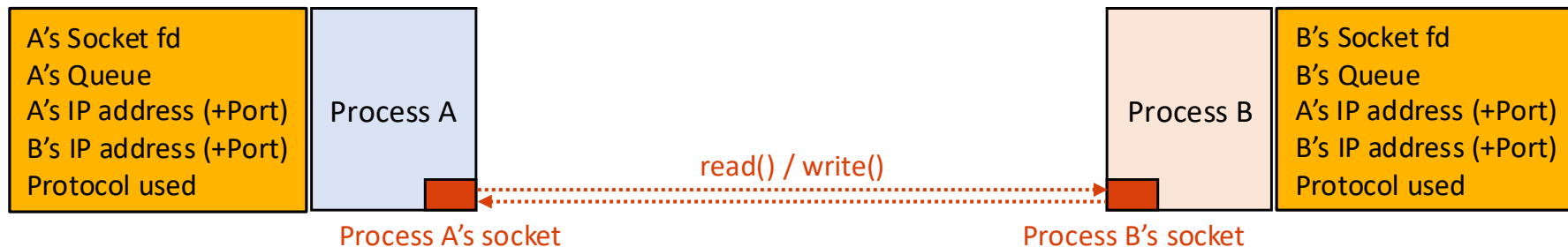
- Socket
 - **Definition:** an *abstract* structure for sending and receiving data
 - **TL; DR:** a *bi-directional* pipe
- Socket components
 - A structure (① a file descriptor and ② a queue)
 - IP addresses (③ source and ④ destination addresses)
 - ⑤ Protocols (*e.g.*, TCP/IP or UDP) to use

PROVIDE ABSTRACTION: TCP VS. UDP

- Socket
 - Provide two communication models
 - SOCK_STREAM: TCP (transmission control protocol)
 - Connection-oriented (3-way handshake)
 - Guarantees in-order, reliable delivery
 - Re-send upon packet losses
 - ex. HTTP, SSH, ...
 - SOCK_DGRAM: UDP (user datagram protocol)
 - Connectionless
 - Low latency, but no reliability or ordering guarantees
 - ex. DNS, Streaming, Game servers, ... (speed > reliability)

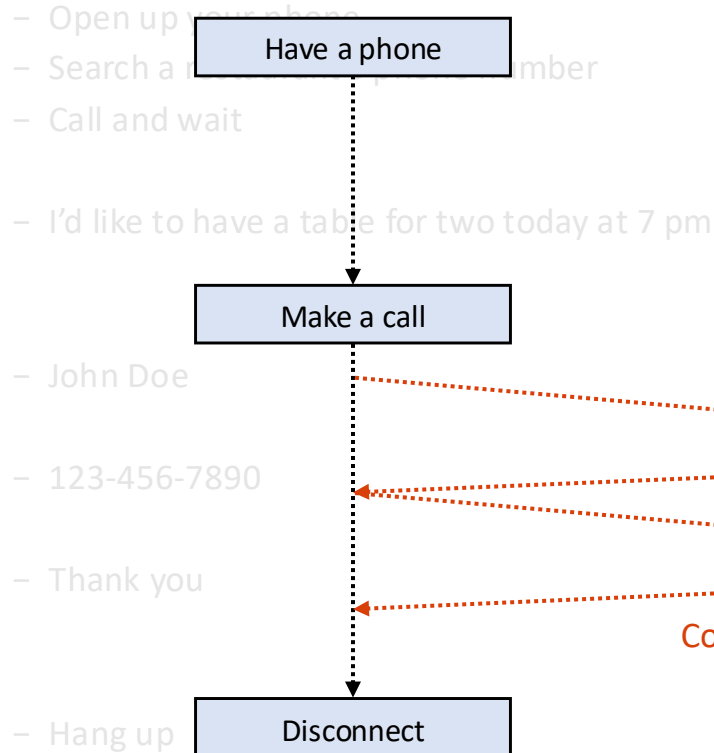
PROVIDE ABSTRACTION: SOCKET

- Socket
 - **Definition:** an *abstract* structure for sending and receiving data
 - **TL; DR:** a *bi-directional* pipe
- Socket components
 - A structure (① a file descriptor and ② a queue)
 - IP addresses (③ source and ④ destination addresses)
 - ⑤ Protocols (*e.g.*, TCP/IP or UDP) to use

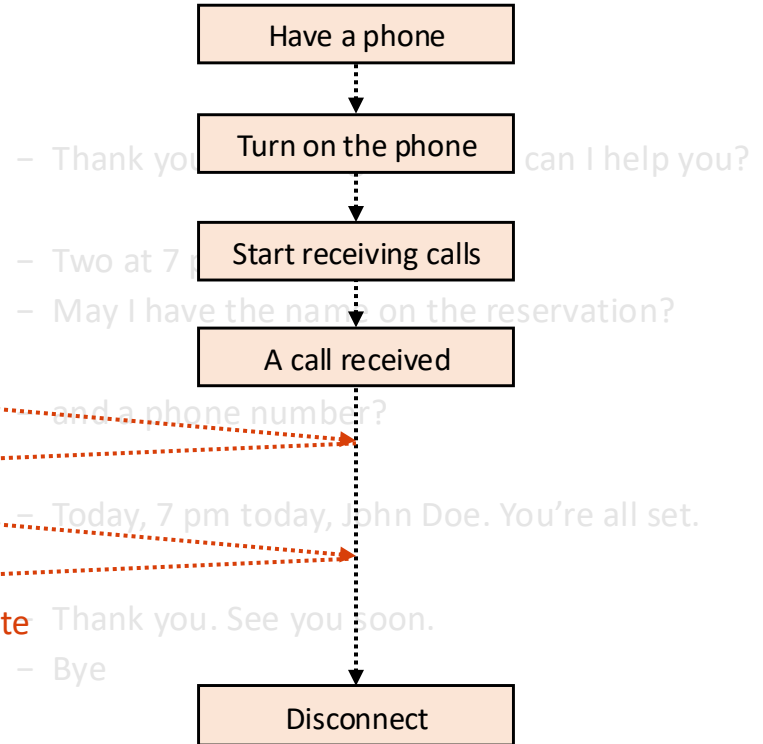


PROVIDE ABSTRACTION: SOCKET PROGRAMMING

- Caller (You: **client**)



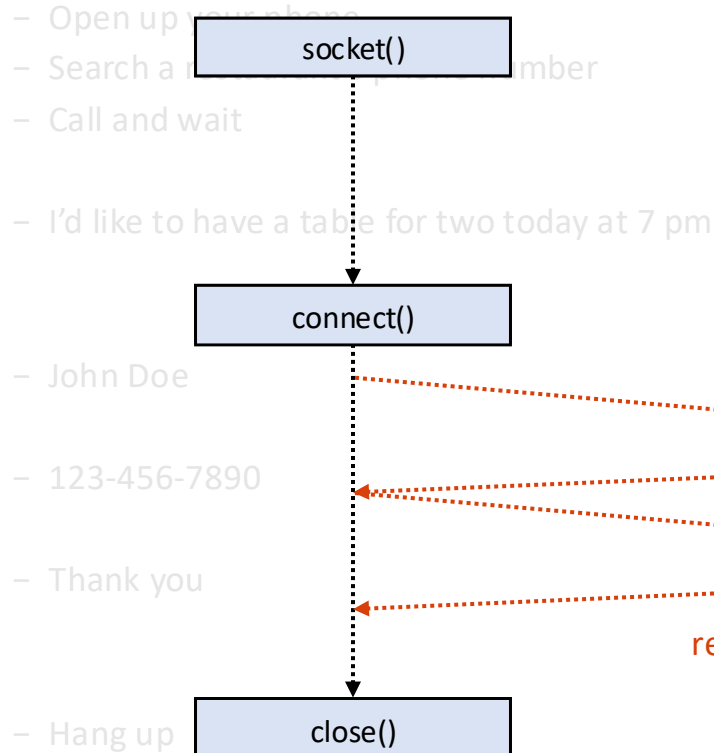
- Callee (XYZ Restaurant: **server**)



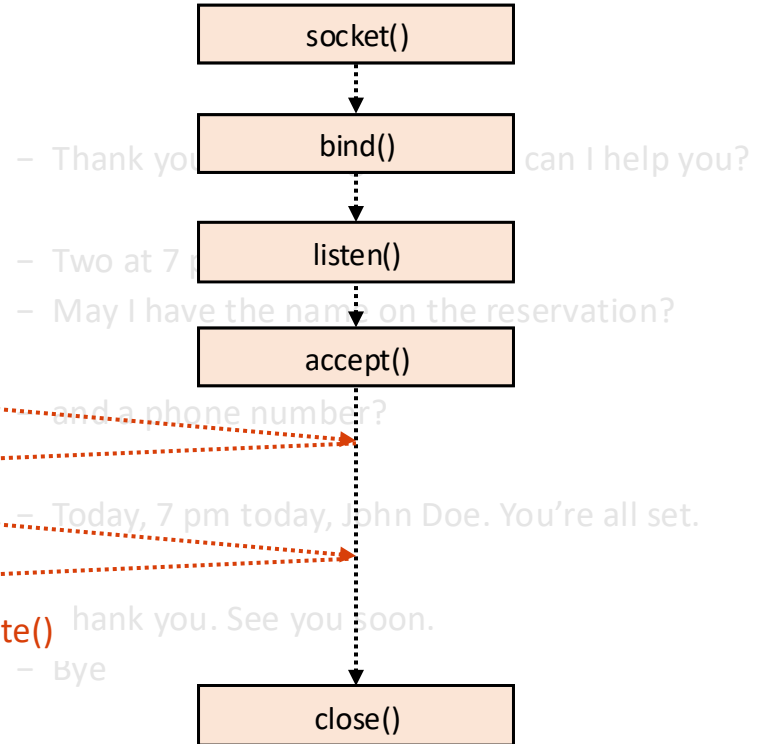
Communicate

PROVIDE ABSTRACTION: SOCKET PROGRAMMING

- Caller (You: **client**)

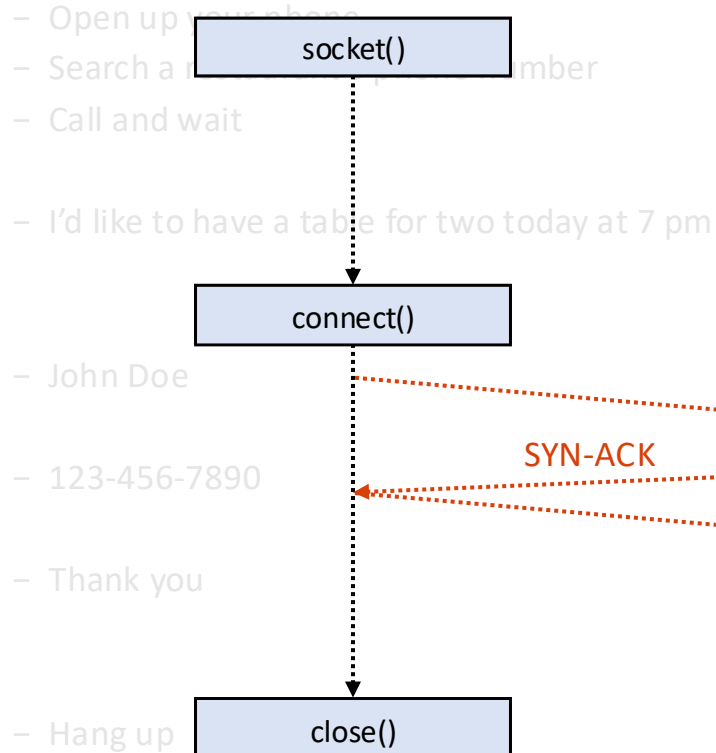


- Callee (XYZ Restaurant: **server**)

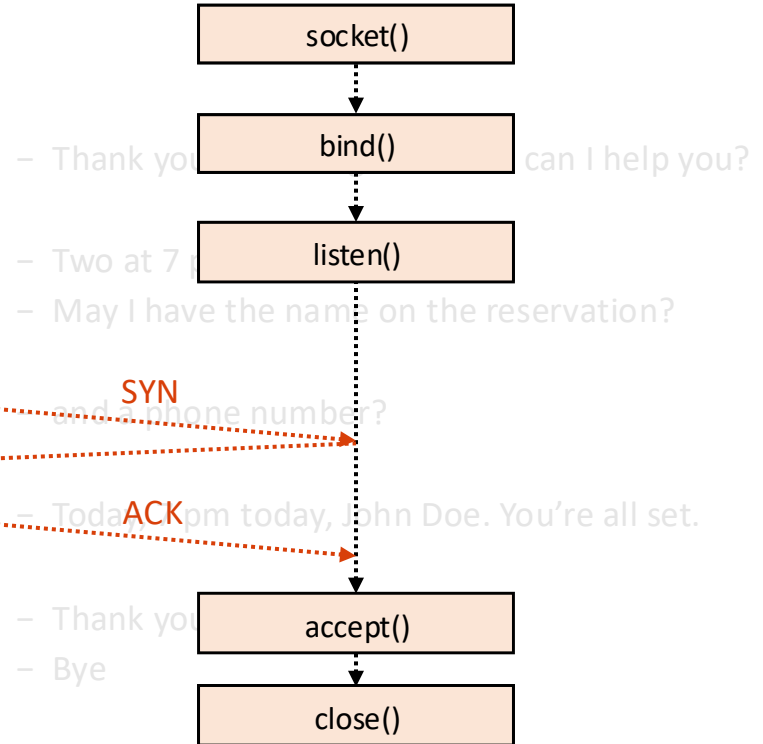


PROVIDE ABSTRACTION: 3-WAY HANDSHAKE

- Caller (You: **client**)



- Callee (XYZ Restaurant: **server**)



PROVIDE ABSTRACTION: 3-WAY HANDSHAKE

- What happens between `connect()` and `accept()`?
 - TCP establishes a connection via *3-way handshake*
 - OS kernel
 - SYN received : OS queues the connection, `listen(fd, backlog)`
 - Backlog : max number of completed connections not yet `accept()`-ed
 - Handshake-n : `accept()` returns a new fd
 - Users
 - `connect()` triggers SYN automatically (kernel handles it)
 - `accept()` returns only after handshake is complete
 - Note
 - socket fd (listening) != new fd returned by `accept()`
 - Design choice: separate “waiting for connections” from “communicating with a client”

TOPICS FOR TODAY

- Part III: Sockets

- Provide abstraction

- Why is PIPE-based IPC not enough?
 - What is a socket?

- Offer standard interface

- How do we use sockets?
 - How do we handle multiple concurrent connections?

- Manage resources

- (Not in this lecture) How does the OS manages socket internals?

OFFER STANDARD INTERFACE: SOCKET

- Socket server-side system calls
 - `int` `socket(int domain, int type, int protocol);`
 - `int` `setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`
 - `int` `bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
 - `int` `listen(int sockfd, int backlog);`
 - `int` `accept(int sockfd, struct sockaddr *restrict addr, socklen_t *restrict addrlen);`

OFFER STANDARD INTERFACE: SOCKET

- Socket server-side system calls
 - `int socket(int domain, int type, int protocol);`
 - `int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`
 - `int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
 - `int listen(int sockfd, int backlog);`
 - `int accept(int sockfd, struct sockaddr *restrict addr, socklen_t *restrict addrlen);`

Domain	Descriptions
AF_UNIX	local communication*
AF_LOCAL	synonym for AF_UNIX
AF_INET	IPv4 Internet protocol*
AF_INET6	IPv6 Internet protocol
AF_PACKET	low-level communication protocol
...	

Type	Descriptions
SOCK_STREAM	TCP (reliable, ordered)
SOCK_DGRAM	UDP (fast, unreliable)
SOCK_RAW	raw network protocol access
...	

Protocol	0 (auto-selected, always use this)
-----------------	------------------------------------

OFFER STANDARD INTERFACE: SERVER.C

... omit the includes

```
#define BUF_SIZE      1024
#define PORT          8080
```

```
int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello (server)!";
```

AF_INET (IPv4)
SOCK_STREAM (bi-directional)

```
// create socket (returns a sockfd for reading/writing)
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}
```

OFFER STANDARD INTERFACE: SOCKET

- Socket server-side system calls

- `int socket(int domain, int type, int protocol);`
- `int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`
- `int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
- `int listen(int sockfd, int backlog);`
- `int accept(int sockfd, struct sockaddr *restrict addr, socklen_t *restrict addrlen);`

Level	Descriptions
SOL_SOCKET	*to set the socket option
IPPROTO_TCP	to interpret the option as TCP
...	

Option Name	Descriptions
SO_REUSEADDR	*use the address in “time-wait”
SO_REUSEPORT	*use the port in “time-wait”
SO_DEBUG	turn on recording of debug info
SO_BROADCAST	broadcast messages (UDP)
SO_KEEPALIVE	keeps connection alive
...	

- `SO_REUSEADDR` | `SO_REUSEPORT`; otherwise need to wait 60 seconds before each run

OFFER STANDARD INTERFACE: SERVER.C

... omit the includes

```
#define BUF_SIZE      1024
#define PORT          8080
```

```
int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello (server)!";
```

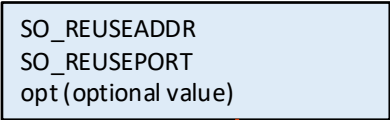
```
// create socket (returns a sockfd for reading/writing)
```

```
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}
```

```
// configure the socket by setting the options
```

```
if (setsockopt(server_fd, SOL_SOCKET,
    SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}
```

SO_REUSEADDR
SO_REUSEPORT
opt (optional value)



OFFER STANDARD INTERFACE: SOCKET

- Socket server-side system calls

- `int socket(int domain, int type, int protocol);`
- `int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`
- `int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
- `int listen(int sockfd, int backlog);`
- `int accept(int sockfd, struct sockaddr *restrict addr, socklen_t *restrict addrlen);`

Argument	Descriptions
addr	IPv4/v6 address structure
addrlen	“sizeof” the above structure
...	

OFFER STANDARD INTERFACE: SERVER.C

Bind the socket to the address
> Any IP (of the host)
> Port # 8080

... omit the includes

```
#define BUF_SIZE      1024
#define PORT          8080
```

```
int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello (server)!";
```

// create socket (returns a sockfd for reading/writing)

```
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}
```

// configure the socket by setting the options

```
if (setsockopt(server_fd, SOL_SOCKET,
    SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}
```

```
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY; // bind to any address
address.sin_port = htons(PORT);       // format the port num
```

```
// attach socket to the port 8080
if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}
```

OFFER STANDARD INTERFACE: SOCKET

- Socket server-side system calls

- `int socket(int domain, int type, int protocol);`
- `int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`
- `int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
- **`int listen(int sockfd, int backlog);`**
- `int accept(int sockfd, struct sockaddr *restrict addr, socklen_t *restrict addrlen);`

Argument	Descriptions
backlog	max number of waiting connections
...	

OFFER STANDARD INTERFACE: SERVER.C

... omit the includes

```
#define BUF_SIZE      1024
#define PORT          8080
```

```
int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello (server)!";

    // create socket (returns a sockfd for reading/writing)
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // configure the socket by setting the options
    if (setsockopt(server_fd, SOL_SOCKET,
        SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }
}
```

```
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY; // bind to any address
address.sin_port = htons(PORT);       // format the port num
```

```
// attach socket to the port 8080
if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}
```

```
if (listen(server_fd, 3) < 0) {
    perror("listen failed");
    exit(EXIT_FAILURE);
}
```

Listen incoming connections
> Use the socket fd
> Allow 3 connections (max.)

OFFER STANDARD INTERFACE: SOCKET

- Socket server-side system calls

- `int socket(int domain, int type, int protocol);`
- `int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`
- `int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
- `int listen(int sockfd, int backlog);`
- `int accept(int sockfd, struct sockaddr *restrict addr, socklen_t *restrict addrlen);`

Argument	Descriptions
addr	IPv4/v6 address structure (client)
addrlen	“sizeof” the above structure (client)
...	

OFFER STANDARD INTERFACE: SERVER.C

... omit the includes

```
#define BUF_SIZE      1024
#define PORT          8080

int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello (server)!";

    // create socket (returns a sockfd for reading/writing)
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // configure the socket by setting the options
    if (setsockopt(server_fd, SOL_SOCKET,
        SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }
}
```

```
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY; // bind to any address
address.sin_port = htons(PORT);       // format the port num

// attach socket to the port 8080
if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

if (listen(server_fd, 3) < 0) {
    perror("listen failed");
    exit(EXIT_FAILURE);
}

if ((new_socket = accept(server_fd,
    (struct sockaddr*)&address,
    (socklen_t*)&sizeof(address))) < 0) {
    perror("accept");
    exit(EXIT_FAILURE);
}

valread = read(new_socket, buffer, 1024);
printf("%s\n", buffer);
send(new_socket, hello, strlen(hello), 0);
printf("Message sent (server)\n");
return 0;
}
```

Start accepting connections
> Use the socket fd
> Use the address specified
> Return the fd (accepted)

OFFER STANDARD INTERFACE: SERVER.C

Bind the socket to the address
> Any IP (of the host)
> Port # 8080

... omit the includes

```
#define BUF_SIZE      1024
#define PORT          8080
```

```
int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello (server)!";
```

AF_INET (IPv4)
SOCK_STREAM (bi-directional)

SO_REUSEADDR
SO_REUSEPORT
opt (optional value)

```
// create socket (returns a sockfd for reading/writing)
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}
```

```
// configure the socket by setting the options
if (setsockopt(server_fd, SOL_SOCKET,
               SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}
```

```
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY; // bind to any address
address.sin_port = htons(PORT);       // format the port num
```

```
// attach socket to the port 8080
if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}
```

```
if (listen(server_fd, 3) < 0) {
    perror("listen failed");
    exit(EXIT_FAILURE);
}
```

Listen incoming connections
> Use the socket fd
> Allow 3 connections (max.)

```
if ((new_socket = accept(server_fd,
                        (struct sockaddr*)&address,
                        (socklen_t*)&sizeof(address)) < 0) {
    perror("accept");
    exit(EXIT_FAILURE);
}
```

```
valread = read(new_socket, buffer, 1024);
printf("%s\n", buffer);
send(new_socket, hello, strlen(hello), 0);
printf("Message sent (server)\n");
return 0;
```

Start accepting connections
> Use the socket fd
> Use the address specified
> Return the fd (accepted)

```
}
```


OFFER STANDARD INTERFACE: SERVER.C

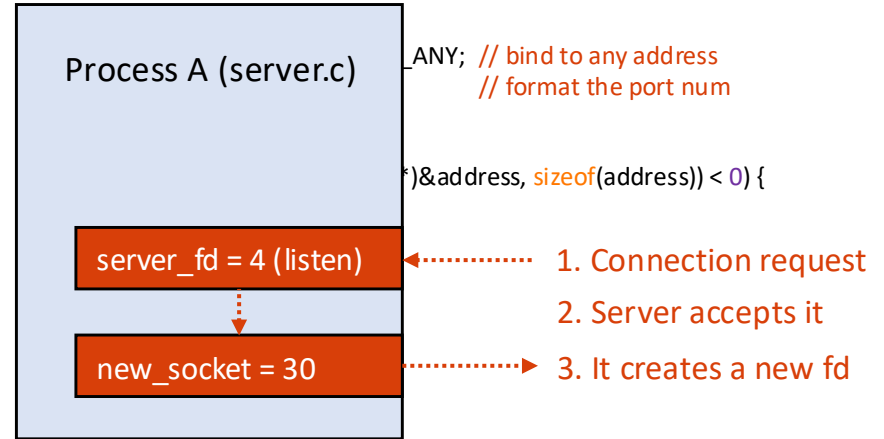
... omit the includes

```
#define BUF_SIZE      1024
#define PORT          8080
```

```
int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello (server)!";
```

```
// create socket (returns a sockfd for reading/writing)
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}
```

```
// configure the socket by setting the options
if (setsockopt(server_fd, SOL_SOCKET,
    SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}
```



```
if ((new_socket = accept(server_fd,
    (struct sockaddr*)&address,
    (socklen_t*)&sizeof(address))) < 0) {
    perror("accept");
    exit(EXIT_FAILURE);
}
```

```
valread = read(new_socket, buffer, 1024);
printf("%s\n", buffer);
send(new_socket, hello, strlen(hello), 0);
printf("Message sent (server)\n");
return 0;
```

Start accepting connections
> Use the socket fd
> Use the address specified
> Return the fd (accepted)

OFFER STANDARD INTERFACE: SERVER.C

socket fd != new_socket

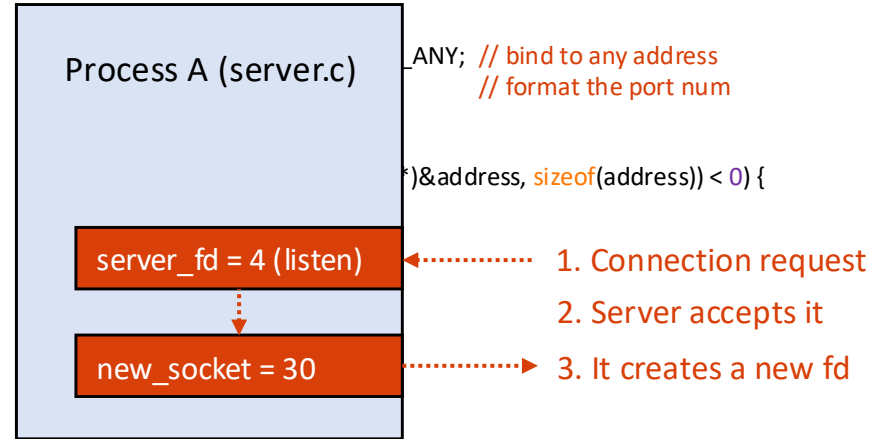
Design choice:

We want to *separate* the file descriptor for listening connection requests (socket_fd) from the file descriptor used for communicating with the client (new_socket)

```
char buffer[BUF_SIZE] = { 0 };
char* hello = "Hello (server)!";
```

```
// create socket (returns a sockfd for reading/writing)
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}
```

```
// configure the socket by setting the options
if (setsockopt(server_fd, SOL_SOCKET,
    SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}
```



```
if ((new_socket = accept(server_fd,
    (struct sockaddr*)&address,
    (socklen_t*)&sizeof(address))) < 0) {
    perror("accept");
    exit(EXIT_FAILURE);
}
```

```
valread = read(new_socket, buffer, 1024);
printf("%s\n", buffer);
send(new_socket, hello, strlen(hello), 0);
printf("Message sent (server)\n");
return 0;
```

Start accepting connections

- > Use the socket fd
- > Use the address specified
- > Return the fd (accepted)

OFFER STANDARD INTERFACE: SOCKET

- Socket client-side system calls
 - `int` `socket(int domain, int type, int protocol);`
 - `int` `setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`
 - `int` `connect(int sockfd, struct sockaddr *restrict addr, socklen_t addrlen);`

Argument	Descriptions
<code>addr</code>	IPv4/v6 address structure (client)
<code>addrlen</code>	“sizeof” the above structure (client)
...	

OFFER STANDARD INTERFACE: CLIENT.C

```
#define IPADDR "127.0.0.1"
#define PORT 8080
#define BUFSIZE 1024
```

AF_INET (IPv4)
SOCK_STREAM (bi-directional)

```
int main(void)
{
    int sock = 0, valread;
    struct sockaddr_in serv_addr;
    char* hello = "Hello (client)";
    char buffer[BUFSIZE] = { 0 };
```

```
// create a socket
if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    printf("Error: socket creation error\n");
    return -1;
}
```

```
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);
```

```
// convert IP addresses from text to binary
if (inet_pton(AF_INET, IPADDR, &serv_addr.sin_addr) <= 0) {
    printf("Error: invalid address, address not supported\n");
    return -1;
}
```

```
if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("Connection Failed\n");
    return -1;
}
```

```
send(sock, hello, strlen(hello), 0);
printf("Message sent (client)\n");
valread = read(sock, buffer, BUFSIZE);
printf("%s\n", buffer);
```

```
return 0;
}
```

Connect to the server, running on
the IP address we specify "127.0.0.1"

Execution result

```
$ gcc -o server server.c
$ gcc -o client client.c
$ ./server &
$ ./client
```

```
Message sent (client)
Hello (client)
Message sent (server)
Hello (server)
```

SOCKET PROGRAMMING EXAMPLE: UNIX DOMAIN SOCKET

- Unix domain socket (AF_UNIX)
 - **Definition:** a socket that communicates between processes on the same host
 - **Properties**
 - Faster than AF_INET, as AF_UNIX does not go through the network stack
 - Use a file system path as an “address” instead of IP + Port
 - **Real-world examples**
 - Docker : /var/run/docker.sock
 - MySQL : /tmp/mysql.lock
 - SSH agent : \$SSH_AUTH_SOCK
 - ... (most linux daemons)

UNIX DOMAIN SOCKET: SERVER.C

Bind the socket to the address
> SOCKET_PATH

... omit the includes

```
#define SOCKET_PATH /tmp/myapp.sock
#define BUF_SIZE 1024
```

```
int main(void) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    char buffer[BUF_SIZE] = { 0 };
    char* hello = "Hello from server!";
```

AF_INET (IPv4)

```
// create socket (returns a sockfd for reading/writing)
if ((server_fd = socket(AF_UNIX, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}
```

```
// configure the socket by setting the options
if (setsockopt(server_fd, SOL_SOCKET,
    SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt))) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}
```

```
memset(&address, 0, sizeof(address));
address.sun_family = AF_UNIX;
strcpy(address.sun_path, SOCKET_PATH)
```

```
// attach socket to the port a file
if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}
```

```
if (listen(server_fd, 3) < 0) {
    perror("listen failed");
    exit(EXIT_FAILURE);
}
```

```
if ((new_socket = accept(server_fd,
    (struct sockaddr*)&address,
    (socklen_t*)&sizeof(address))) < 0) {
    perror("accept");
    exit(EXIT_FAILURE);
}
```

```
valread = read(new_socket, buffer, 1024);
printf("%s\n", buffer);
send(new_socket, hello, strlen(hello), 0);
printf("Message sent (server)\n");
return 0;
```

```
}
```

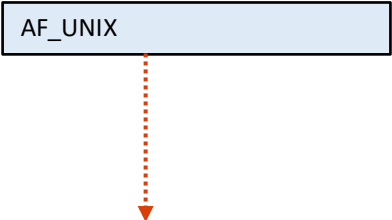
UNIX DOMAIN SOCKET: CLIENT.C

```
#define SOCKET_PATH /tmp/myapp.sock
#define BUFSIZE 1024
```

```
int main(void)
```

```
{
    int sock = 0, valread;
    struct sockaddr_un serv_addr;
    char* hello = "Hello from client";
    char buffer[BUFSIZE] = { 0 };
```

AF_UNIX



```
// create a socket
if ((sock = socket(AF_UNIX, SOCK_STREAM, 0)) < 0) {
    printf("Error: socket creation error\n");
    return -1;
}
```

```
memset(&address, 0, sizeof(address))
address.sun_family = AF_UNIX;
strcpy(address.sun_path, SOCKET_PATH);
```

```
// convert IP addresses from text to binary
if (inet_pton(AF_INET, IPADDR, &serv_addr.sin_addr) <= 0) {
    printf("Error: invalid address, address not supported\n");
    return -1;
}
```

```
if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("Connection Failed\n");
    return -1;
}
```

```
send(sock, hello, strlen(hello), 0);
printf("Message sent client\n");
valread = read(sock, buffer, BUFSIZE);
printf("%s\n", buffer);
```

```
return 0;
}
```

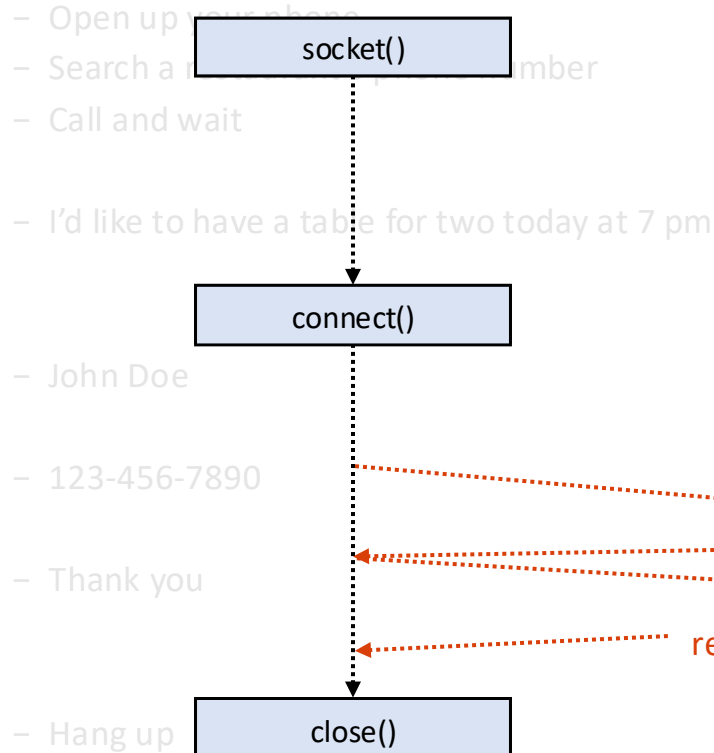
Execution result

```
$ gcc -o server server.c
$ gcc -o client client.c
$ ./server &
$ ./client
```

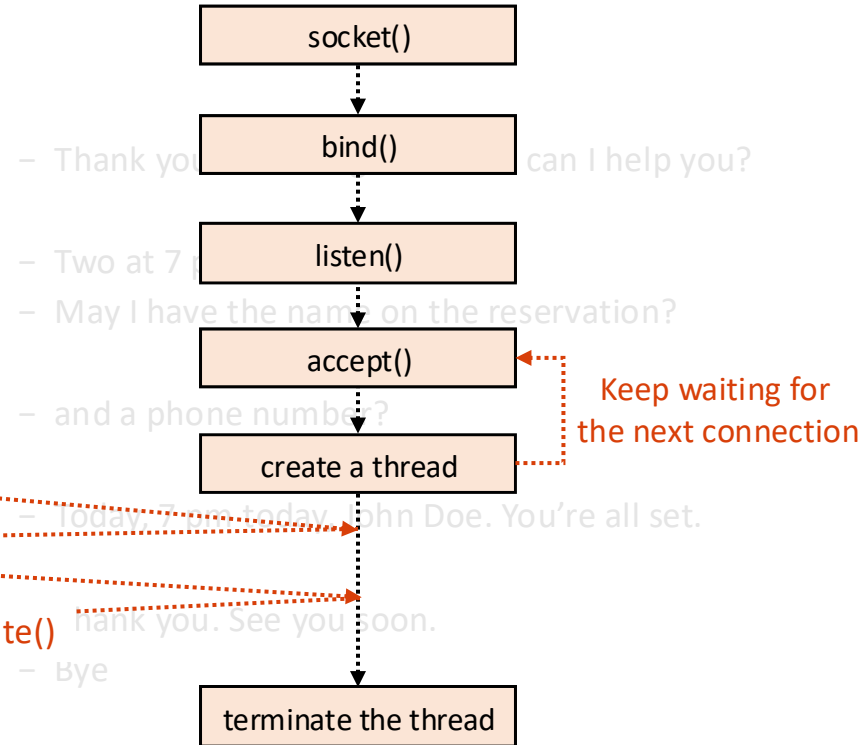
```
Message sent client
Hello from client
Message sent server
Hello from server
```

SOCKET PROGRAMMING EXAMPLE: MULTI-THREADED SERVER

• Caller (Client)



• Callee (Server)



SOCKET PROGRAMMING EXAMPLE: MULTI-THREADED SERVER

... omit the code for creating and binding sockets ...

```
if (listen(server_fd, 100) < 0) {  
    perror("listen failed");  
    exit(EXIT_FAILURE);  
}
```

Listen; up to 100 connections

Keep waiting

Once csocket is greater than 0,
then it proceeds to the next line

```
pthread_t tid;  
while (1) {  
    if ((csocket = accept(server_fd,  
                          (struct sockaddr *)&client,  
                          (socklen_t *)&c) < 0) {  
        printf("Server waits for a connection\n");  
        continue;  
    }  
}
```

```
printf("Server accepts the connection\n");  
int *new_sock = malloc(sizeof(int)); *new_sock = csocket;
```

```
if (pthread_create(&tid, NULL, conn_handler, (void *)new_sock) < 0) {  
    perror("Error: cannot start a thread");  
    exit(EXIT_FAILURE);  
}
```

```
pthread_detach(tid);  
return 0; // this thread infinitely runs, this line won't be reached
```

```
void *conn_handler(void *arg) {  
    int rlen;  
    int socket_desc = *(int *)arg; free(arg);  
    char buffer[BUF_SIZE] = { 0 };  
    char *ack = "Received";
```

```
while ((rlen = read(socket_desc, buffer, BUF_SIZE)) > 0) {  
    buffer[rlen] = '\0';  
    if (rlen < 0) continue;  
  
    printf("(server) Received: %s\n", buffer);  
    send(socket_desc, ack, strlen(ack), 0);  
    printf("(server) Ack sent\n");  
}
```

```
close(socket_desc);  
return 0;  
}
```

Create a thread that handles the
communications between this
server and the connected client
(we pass socket_desc as an arg)

Keep reading data from the socket

1. If no data, it continues
2. If data is, it prints out the data
3. Then it sends the ack message

TOPICS FOR TODAY

- Part III: Sockets
 - Provide abstraction
 - Why is PIPE-based IPC not enough?
 - What is a socket?
 - Offer standard interface
 - How do we use sockets?
 - How do we handle multiple concurrent connections?
 - Manage resources
 - (Not in this lecture) How does the OS manages socket internals?

Thank You!

M/W 12:00 – 1:50 PM (LINC #200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI